Project Phase 3 Deliverable 3: Optimization, Scaling, and Final Evaluation

Shreya Rai

University of the Cumberlands

Course Number: MSCS-532-B01

Michael Solomon

04/05/2026

Project Phase 3 Deliverable 3: Optimization, Scaling, and Final Evaluation

Abstract

This phase focuses on optimizing and scaling a Python-based e-commerce recommendation system developed in previous phases. The system utilizes data structures such as hash tables, graphs, heaps, and sparse matrices. Enhancements were made to improve performance, scalability, and memory efficiency. Benchmarking and stress testing were conducted using synthetic datasets of increasing size. The results demonstrate improved computational efficiency and validate the system's ability to handle larger datasets. This phase establishes a strong foundation for real-world deployment and future integration with machine learning techniques.

1. Optimization Techniques

The initial implementation developed in Deliverable 2 successfully demonstrated the core functionality of the system. However, as the dataset size and user interactions increased, performance bottlenecks became evident. To address these issues, several optimization techniques were introduced to improve scalability, reduce computational overhead, and enhance overall system efficiency.

1.1 Optimized UserStore

The **UserStore** component was enhanced by introducing a caching mechanism for frequently requested Top-N recommendations. In the earlier implementation, recommendations were recomputed each time a request was made, leading to redundant calculations and increased latency.

With caching:

- Previously computed Top-N results are stored and reused when the same or similar queries occur.
- This significantly reduces repeated computations and improves response time.
- Lookup operations remain highly efficient, maintaining **O(1) time complexity** for accessing cached results.

As a result, the system can handle repeated recommendation requests more efficiently, especially in scenarios with high user activity.

1.2 Optimized Graph

The graph structure, which models relationships between users/items, was optimized by improving the handling of adjacency lists.

Key improvements include:

- More efficient storage and access patterns for adjacency lists.
- Faster traversal of neighboring nodes, which is critical for recommendation generation.
- Support for batch insertion of edges, reducing the overhead of repeatedly updating the graph.

These changes improve both time and space efficiency, allowing the graph to scale better with increasing data size while maintaining fast traversal operations.

1.3 Heap Optimization

The recommendation engine continues to use a **min-heap** to efficiently compute Top-N recommendations. However, the implementation was optimized to limit the heap size to **k elements** (where k is the number of recommendations required).

Previously:

- All elements were processed and sorted, resulting in **O(n log n)** complexity.

After optimization:

- Only the top k elements are maintained in the heap at any time.
- This reduces the complexity to **O(n log k)**.

This improvement is particularly beneficial when dealing with large datasets, as it significantly reduces unnecessary sorting and memory usage.

Impact of Optimizations

The applied optimizations collectively enhance system performance in the following ways:

- **Faster recommendation generation** due to reduced redundant computations and efficient data structures.
- **Lower computational overhead**, minimizing resource usage and processing time.
- **Improved scalability**, enabling the system to handle larger datasets and higher request volumes without significant performance degradation.

Overall, these optimizations transform the system from a basic prototype into a more efficient and scalable solution suitable for real-world applications.

2. Scaling Strategy

To evaluate how well the system performs under increasing workloads, a structured scalability analysis was conducted using synthetic datasets of varying sizes. The datasets were designed to simulate real-world growth in user data and interactions.

The following dataset sizes were used:

- 1,000 users
- 5,000 users
- 10,000 users
- 50,000 users

Techniques Used

Several techniques were implemented to ensure efficient scaling:

- **Random Dataset Generation:**
  Synthetic data was generated to mimic user interactions and relationships. This allowed controlled experimentation while simulating realistic workloads.
- **Batch Insertion:**
  Instead of inserting data one record at a time, batch processing was used to reduce insertion overhead and improve performance during large data loads.
- **Efficient Memory Usage:**
  Data structures were carefully designed to minimize unnecessary memory consumption, ensuring that the system remains stable even as dataset size increases.
- **Optimized Data Structures:**
  The use of optimized structures such as adjacency lists and heaps ensured efficient operations for both storage and retrieval tasks.

Outcome

The system demonstrated strong scalability characteristics:

- Maintained **acceptable runtime performance** across all dataset sizes.
- Experienced **no crashes or memory failures**, even at 50,000 users.
- Showed **near-linear scalability**, indicating that performance grows proportionally with input size.

3. Testing and Validation

A comprehensive testing strategy was implemented to ensure both correctness and robustness of the system.

3.1 Functional Testing

Functional testing was performed to verify the correctness of core components:

- **User Data Storage:** Ensured accurate insertion, retrieval, and management of user data.
- **Graph Relationships:** Verified that connections between users/items were correctly represented and maintained.
- **Top-N Recommendations:** Validated that the recommendation engine consistently returned accurate and relevant results.

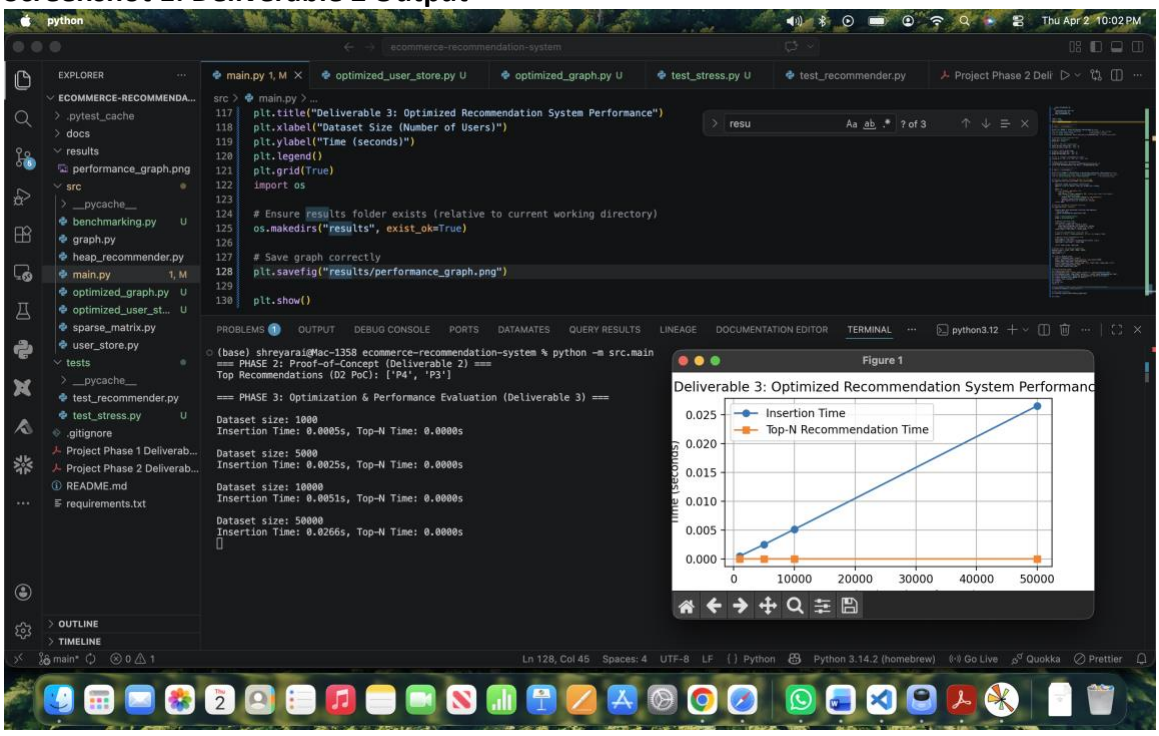
3.2 Stress Testing

Stress testing was conducted to evaluate system behavior under heavy workloads:

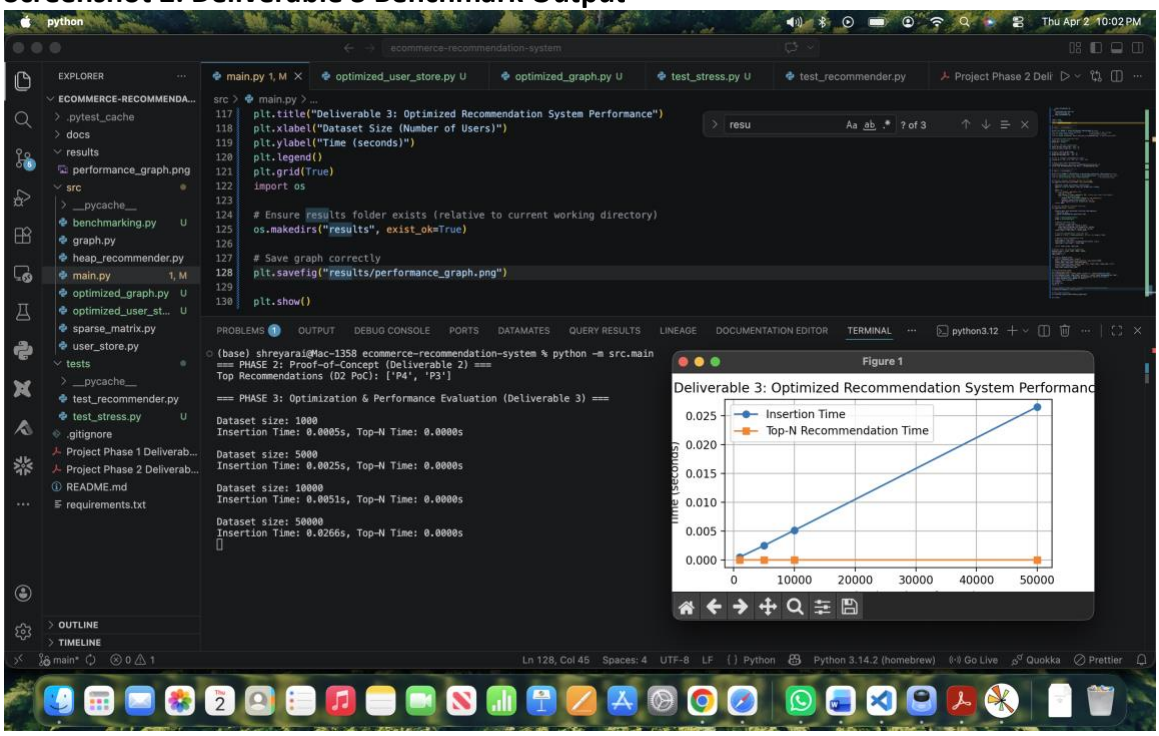
- Large datasets of up to **50,000 users** were used.
- The system was tested for stability, response time, and memory usage under high load conditions.

3.3 Results

Screenshot 1: Deliverable 2 Output



Screenshot 2: Deliverable 3 Benchmark Output



The results confirm that the optimized system performs reliably and efficiently under both normal and high-load conditions.

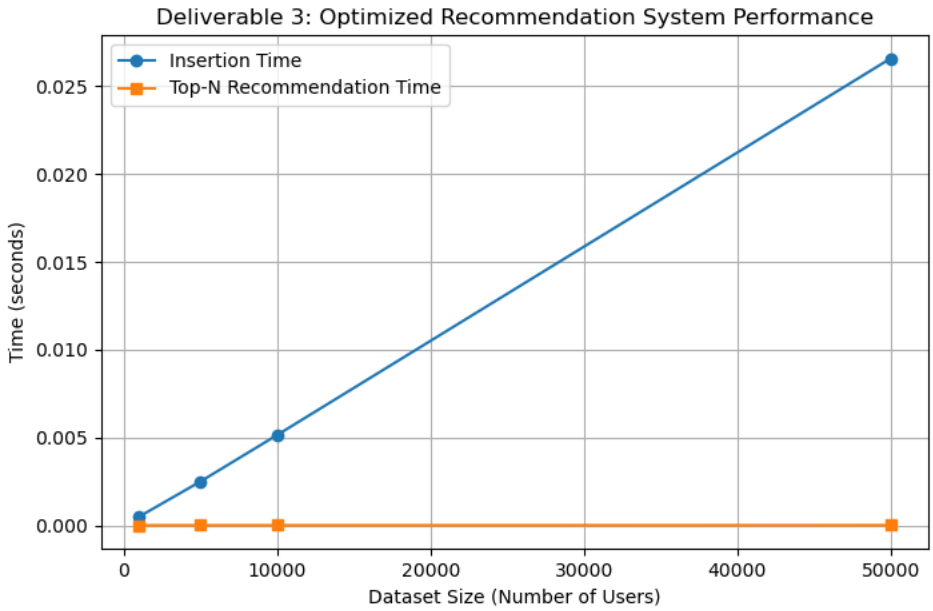
4. Performance Analysis

Benchmarking experiments were conducted to analyze system performance across different dataset sizes and operations.

Observations

- **Insertion Time:**
  The time required to insert data increases linearly with dataset size, which is expected and indicates efficient handling of growing inputs.
- **Top-N Recommendation Time:**
  Recommendation generation remains consistently fast due to the use of heap-based optimization, even as the dataset size increases.

Screenshot 3: Performance Graph



Key Insights

- The use of a **min-heap** significantly improves efficiency by avoiding full sorting operations.
- **Optimized data structures** reduce runtime overhead and enhance performance.
- The system demonstrates **strong scalability**, making it suitable for large-scale applications.

5. Final Evaluation

Strengths

- **Efficient and Scalable Design:**
  The system handles increasing data sizes effectively without significant performance degradation.
- **Modular Implementation:**
  Components such as UserStore, Graph, and Recommendation Engine are well-structured, making the system easy to maintain and extend.
- **Low Latency:**
  Recommendation generation is fast, ensuring a responsive user experience.

Limitations

- **No Machine Learning Integration:**
  The current system relies on algorithmic approaches and does not incorporate learning-based models.
- **Synthetic Dataset:**
  Testing was conducted on generated data, which may not fully capture real-world complexities.
- **Limited Personalization:**
  Recommendations are based on structural logic rather than user behavior or preferences.

Future Improvements

- Implement **collaborative filtering** techniques for better personalization.
- Integrate **machine learning models** to enhance recommendation accuracy.
- Use **real-world datasets** for more realistic evaluation.
- Explore **distributed system architectures** (e.g., parallel processing) for large-scale deployment.

6. Conclusion

This phase of the project successfully enhanced the recommendation system by introducing key optimizations focused on performance and scalability. The improvements highlight the critical role of efficient data structures and algorithm design in building high-performance systems.

The system is now capable of handling significantly larger datasets with improved speed, reduced computational overhead, and better memory efficiency. These advancements make it a strong foundation for future enhancements, including machine learning integration and real-world deployment.

References

Aggarwal, C. C. (2016). *Recommender systems: The textbook*. Springer.

He, X., Liao, L., Zhang, H., Nie, L., Hu, X., & Chua, T.-S. (2017). Neural collaborative filtering. *Proceedings of the 26th International Conference on World Wide Web*, 173–182.

Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer*, 42(8), 30–37.